

LAPORAN PRAKTIK UAS BASIS DATA

Sistem Informasi Pengelolaan Kos



Dosen Pengampu:

Asmunin, S.Kom., M.Kom.

Disusun Oleh:

Nama: Elvira Aprilia

NIM: 25091397082 Kelas:

2025I

PROGRAM STUDI MANAJEMEN INFORMATIKA

FAKULTAS VOKASI

UNIVERSITAS NEGERI SURABAYA

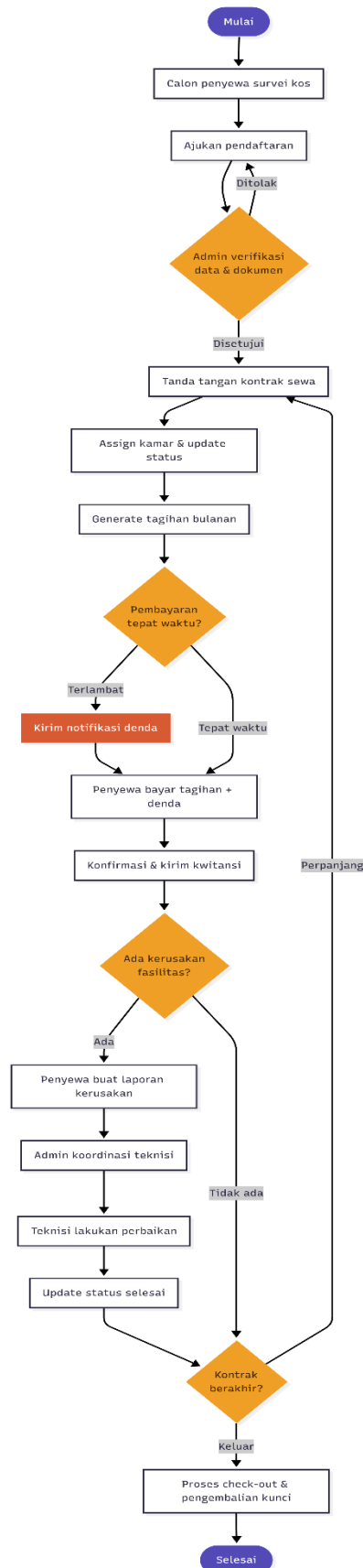
2026

Pendahuluan

Sistem Informasi Pengelolaan Kos adalah sistem berbasis digital yang dirancang untuk mengotomasi dan menyederhanakan seluruh proses operasional pengelolaan rumah kos. Sistem ini mencakup manajemen data penyewa, kamar, pembayaran sewa, pemeliharaan fasilitas, hingga pelaporan keuangan secara terintegrasi.

Tujuan utama pengembangan sistem ini adalah:

- Meningkatkan efisiensi administrasi pengelolaan kos
- Meminimalisir kesalahan pencatatan manual
- Mempercepat proses pembayaran dan konfirmasi
- Menyediakan informasi real-time bagi pemilik dan admin
- Meningkatkan kepuasan penyewa melalui layanan yang lebih terstruktur



1. Proses Bisnis & Modul Sistem

Gambaran Umum Proses Bisnis

Proses bisnis sistem informasi pengelolaan kos dimulai dari calon penyewa yang mengajukan pendaftaran, berlanjut ke pengelolaan kamar, penagihan dan pembayaran sewa secara berkala, penanganan laporan kerusakan fasilitas, hingga proses check-out ketika masa sewa berakhir. Setiap tahapan saling terhubung dan melibatkan beberapa aktor dengan peran yang berbeda.

Alur proses bisnis utama:

Mulai → Survei Kamar → Pendaftaran → Verifikasi Admin → Kontrak Sewa → Assign Kamar → Tagihan Bulanan → Pembayaran → [Ada Kerusakan?] → Pemeliharaan → [Kontrak Berakhir?] → Perpanjang / Check-out → Selesai

Modul-Modul Sistem

Sistem informasi pengelolaan kos terdiri dari 6 modul utama yang saling terintegrasi:

Modul 1 — Pendaftaran Penyewa

Menangani seluruh proses penerimaan penyewa baru, mulai dari pengisian formulir, verifikasi dokumen identitas, hingga penandatanganan kontrak sewa.

- Input data calon penyewa (nama, KTP, pekerjaan, kontak darurat)
- Upload dan verifikasi dokumen pendukung
- Persetujuan atau penolakan permohonan oleh admin
- Pembuatan dan penandatanganan kontrak sewa digital
- Notifikasi status pendaftaran ke calon penyewa

Modul 2 — Manajemen Kamar

Mengelola seluruh data kamar yang tersedia di kos, termasuk status hunian, tipe kamar, harga sewa, dan riwayat penyewa.

- Pendataan kamar beserta tipe dan fasilitas
- Pengaturan harga sewa per kamar
- Update status kamar: tersedia, terisi, atau dalam perbaikan
- Assign kamar ke penyewa yang telah disetujui
- Riwayat penyewa per kamar

Modul 3 — Pembayaran Sewa

Mengotomasi proses penagihan bulanan, konfirmasi pembayaran, perhitungan denda keterlambatan, dan penerbitan kwitansi digital.

- Generate tagihan otomatis sesuai tanggal jatuh tempo kontrak
- Pencatatan pembayaran masuk (manual/transfer)
- Kalkulasi denda otomatis untuk pembayaran terlambat
- Konfirmasi pembayaran oleh admin
- Penerbitan kwitansi digital ke penyewa
- Riwayat transaksi per penyewa

Modul 4 — Pemeliharaan Fasilitas

Menangani pelaporan kerusakan fasilitas oleh penyewa, koordinasi penugasan teknisi, dan pemantauan status perbaikan hingga selesai.

- Form laporan kerusakan (deskripsi, foto, prioritas)
- Review dan validasi laporan oleh admin
- Assign teknisi sesuai jenis kerusakan
- Pemantauan progress: dilaporkan → dikerjakan → selesai
- Pencatatan biaya perbaikan

Modul 5 — Laporan & Analitik

Menyediakan laporan berkala dan analitik operasional bagi pemilik kos dan admin, mencakup laporan pendapatan, tingkat hunian, dan rekap tunggakan.

- Laporan pendapatan bulanan dan tahunan
- Statistik tingkat hunian kamar
- Rekap tunggakan dan penyewa bermasalah
- Laporan biaya pemeliharaan
- Export laporan ke format PDF atau Excel

Modul 6 — Manajemen Akun & Akses

Mengelola data pengguna sistem, pembagian hak akses berdasarkan peran (role), dan pencatatan log aktivitas untuk keamanan sistem.

- Registrasi dan pengelolaan akun pengguna
- Pengaturan role: Pemilik, Admin, Penyewa, Teknisi
- Pembatasan akses fitur berdasarkan role
- Log aktivitas pengguna
- Reset password dan pengelolaan sesi

2. Aktor yang Terlibat per Modul

Sistem ini melibatkan 4 aktor utama dengan peran dan tanggung jawab masing-masing dalam setiap modul.

Aktor	Deskripsi
Pemilik Kos	Pemilik properti, memiliki akses penuh sebagai super admin sistem
Admin	Staf pengelola harian, menangani operasional sistem secara langsung
Penyewa	Penghuni aktif kos, akses terbatas pada data dan transaksi pribadi
Teknisi	Petugas perbaikan, hanya dapat mengakses modul pemeliharaan

Modul 1 — Pendaftaran Penyewa

Aktor	Peran & Tanggung Jawab
Pemilik Kos	Persetujuan akhir untuk penyewa baru, menetapkan kebijakan penerimaan
Admin	Input dan verifikasi data calon penyewa, pembuatan kontrak sewa
Penyewa	Mengisi formulir pendaftaran, melampirkan dokumen, survei kamar
Teknisi	Tidak terlibat

Modul 2 — Manajemen Kamar

Aktor	Peran & Tanggung Jawab
Pemilik Kos	Menetapkan harga kamar, kebijakan fasilitas yang disediakan
Admin	Update status kamar, assign kamar ke penyewa yang disetujui
Penyewa	Memilih dan melihat informasi kamar yang tersedia
Teknisi	Tidak terlibat

Modul 3 — Pembayaran Sewa

Aktor	Peran & Tanggung Jawab
Pemilik Kos	Menerima laporan pendapatan, menetapkan nominal denda keterlambatan

Admin	Konfirmasi pembayaran masuk, cetak kwitansi, rekap tagihan bulanan
Penyewa	Melakukan pembayaran, melihat tagihan dan riwayat transaksi pribadi
Teknisi	Tidak terlibat

Modul 4 — Pemeliharaan Fasilitas

Aktor	Peran & Tanggung Jawab
Pemilik Kos	Menyetujui anggaran perbaikan besar, melihat laporan biaya pemeliharaan
Admin	Menerima dan validasi laporan, assign teknisi, monitoring status pekerjaan
Penyewa	Membuat laporan kerusakan fasilitas, memantau status perbaikan
Teknisi	Menerima tugas perbaikan, melaksanakan pekerjaan, update status selesai

Modul 5 — Laporan & Analitik

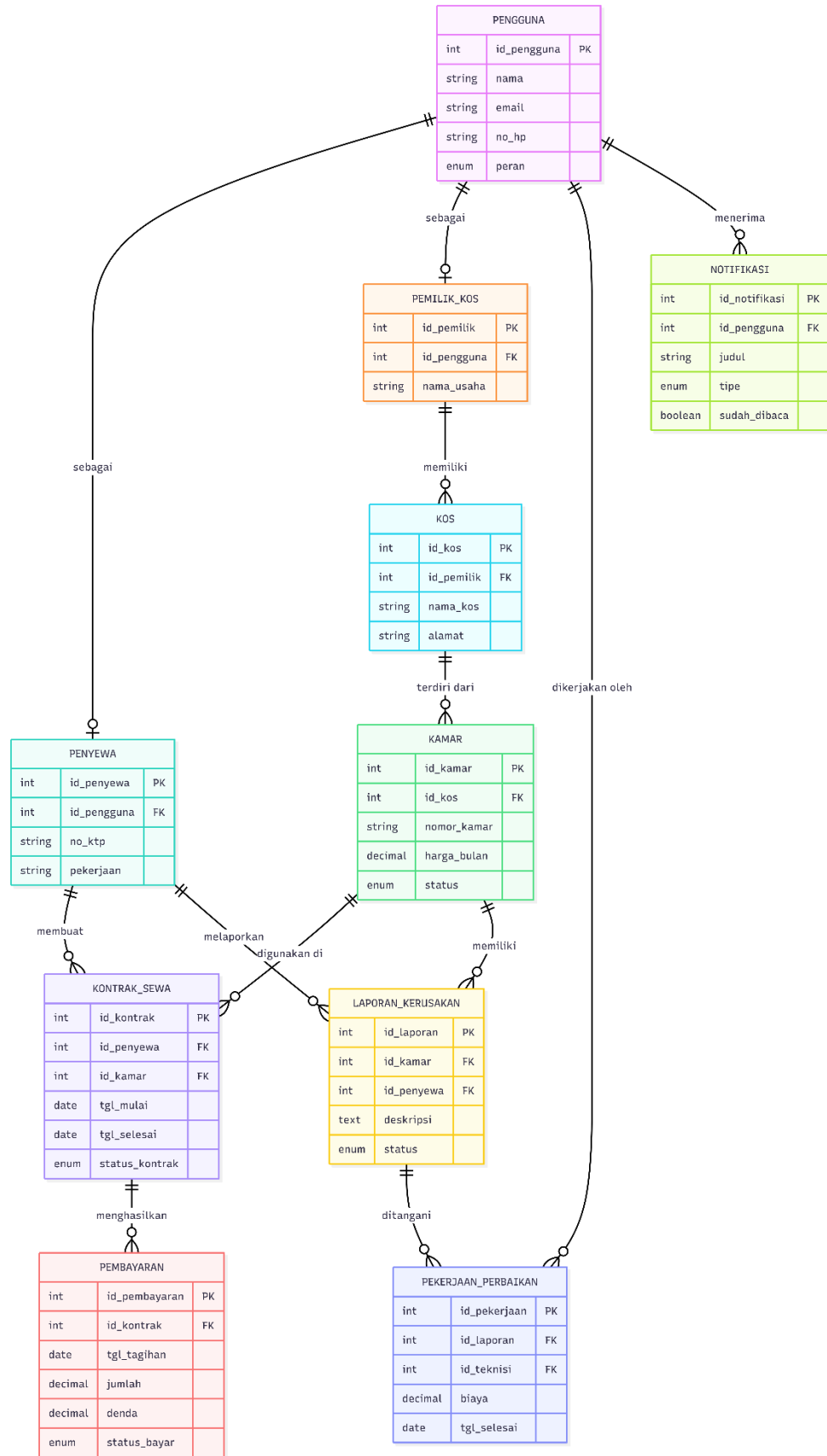
Aktor	Peran & Tanggung Jawab
Pemilik Kos	Membaca laporan keuangan dan operasional bulanan serta tahunan
Admin	Generate laporan, input data untuk rekap operasional
Penyewa	Tidak terlibat
Teknisi	Tidak terlibat

Modul 6 — Manajemen Akun & Akses

Aktor	Peran & Tanggung Jawab
Pemilik Kos	Akses penuh (super admin), mengelola akun admin dan kebijakan sistem
Admin	Mengelola akun penyewa dan teknisi, update data profil
Penyewa	Mengelola profil dan password akun pribadi
Teknisi	Mengelola profil pribadi, akses terbatas sesuai role

3. Entity Relationship Diagram (ERD)

ERD sistem informasi pengelolaan kos terdiri dari 10 entitas utama yang saling berelasi, mencerminkan seluruh proses bisnis dari pendaftaran penyewa hingga pemeliharaan fasilitas.



1. Deskripsi Entitas

PENGGUNA

Entitas pusat yang menyimpan data identitas semua pengguna sistem. Atribut peran membedakan tipe akses: Pemilik, Admin, Penyewa, dan Teknisi. Semua entitas aktor berelasi ke entitas ini melalui foreign key.

Atribut	Tipe	Keterangan
id_pengguna	INT (PK)	Primary key pengguna
nama	VARCHAR	Nama lengkap pengguna
email	VARCHAR	Email unik untuk login
no_hp	VARCHAR	Nomor handphone aktif
peran	ENUM	Pemilik / Admin / Penyewa / Teknisi
dibuat_pada	TIMESTAMP	Waktu akun dibuat

PENYEWA

Ekstensi dari entitas PENGGUNA khusus untuk penyewa aktif. Menyimpan data identitas tambahan seperti nomor KTP dan kontak darurat.

Atribut	Tipe	Keterangan
id_penyewa	INT (PK)	Primary key penyewa
id_pengguna	INT (FK)	Referensi ke tabel PENGGUNA
no_ktp	VARCHAR	Nomor KTP penyewa
pekerjaan	VARCHAR	Pekerjaan/profesi penyewa
kontak_darurat	VARCHAR	Nomor kontak darurat

PEMILIK_KOS

Ekstensi dari entitas PENGGUNA untuk pemilik properti kos. Menyimpan data usaha dan rekening bank untuk keperluan pencatatan keuangan.

Atribut	Tipe	Keterangan
id_pemilik	INT (PK)	Primary key pemilik
id_pengguna	INT (FK)	Referensi ke tabel PENGGUNA
nama_usaha	VARCHAR	Nama usaha/brand kos
alamat_usaha	TEXT	Alamat usaha utama
no_rekening	VARCHAR	Nomor rekening bank pemilik

KOS

Menyimpan data properti kos yang dimiliki oleh pemilik. Satu pemilik dapat memiliki lebih dari satu kos.

Atribut	Tipe	Keterangan
---------	------	------------

id_kos	INT (PK)	Primary key kos
id_pemilik	INT (FK)	Referensi ke tabel PEMILIK_KOS
nama_kos	VARCHAR	Nama/label kos
alamat	TEXT	Alamat lengkap kos
jumlah_kamar	INT	Total unit kamar di kos ini

KAMAR

Menyimpan detail setiap unit kamar dalam satu kos. Atribut status mencatat kondisi kamar:

tersedia, terisi, atau dalam perbaikan.

Atribut	Tipe	Keterangan
id_kamar	INT (PK)	Primary key kamar
id_kos	INT (FK)	Referensi ke tabel KOS
nomor_kamar	VARCHAR	Nomor/kode kamar
tipe	ENUM	Tipe kamar (standar/deluxe/VIP)
harga_per_bulan	DECIMAL	Harga sewa per bulan
status	ENUM	Tersedia / Terisi / Dalam perbaikan
fasilitas	TEXT	Daftar fasilitas yang tersedia

KONTRAK_SEWA

Entitas penghubung antara PENYEWA dan KAMAR, mencatat detail perjanjian sewa termasuk harga yang disepakati dan status kontrak aktif/berakhir.

Atribut	Tipe	Keterangan
id_kontrak	INT (PK)	Primary key kontrak
id_penyewa	INT (FK)	Referensi ke tabel PENYEWA
id_kamar	INT (FK)	Referensi ke tabel KAMAR
tgl_mulai	DATE	Tanggal mulai kontrak
tgl_selesai	DATE	Tanggal berakhir kontrak
harga_disepakati	DECIMAL	Harga sewa yang disepakati
status_kontrak	ENUM	Aktif / Berakhir / Dibatalkan

PEMBAYARAN

Menyimpan setiap transaksi pembayaran sewa per bulan. Berelasi ke KONTRAK_SEWA dan mencatat denda jika pembayaran terlambat.

Atribut	Tipe	Keterangan
id_pembayaran	INT (PK)	Primary key pembayaran

id_kontrak	INT (FK)	Referensi ke tabel KONTRAK_SEWA
tgl_tagihan	DATE	Tanggal tagihan diterbitkan
tgl_bayar	DATE	Tanggal pembayaran dilakukan
jumlah	DECIMAL	Jumlah pembayaran pokok
denda	DECIMAL	Jumlah denda keterlambatan
metode_bayar	ENUM	Transfer / Cash / QRIS
status_bayar	ENUM	Lunas / Belum Bayar / Terlambat

LAPORAN_KERUSAKAN

Mencatat setiap laporan kerusakan fasilitas yang diajukan penyewa. Atribut prioritas membantu admin menentukan urgensi penanganan.

Atribut	Tipe	Keterangan
id_laporan	INT (PK)	Primary key laporan
id_kamar	INT (FK)	Kamar yang mengalami kerusakan
id_penyewa	INT (FK)	Penyewa yang melaporkan
judul	VARCHAR	Judul singkat kerusakan
deskripsi	TEXT	Deskripsi detail kerusakan
prioritas	ENUM	Rendah / Sedang / Tinggi / Darurat
status	ENUM	Dilaporkan / Dikerjakan / Selesai
dilaporkan_pada	TIMESTAMP	Waktu laporan dibuat

PEKERJAAN_PERBAIKAN

Mencatat penugasan dan hasil pekerjaan teknisi untuk setiap laporan kerusakan. Satu laporan dapat memiliki lebih dari satu pekerjaan jika penanganan bertahap.

Atribut	Tipe	Keterangan
id_pekerjaan	INT (PK)	Primary key pekerjaan
id_laporan	INT (FK)	Referensi ke laporan kerusakan
id_teknisi	INT (FK)	Referensi ke PENGGUNA (teknisi)
Atribut	Tipe	Keterangan
tgl_mulai	DATE	Tanggal mulai pengerjaan
tgl_selesai	DATE	Tanggal pekerjaan selesai
biaya	DECIMAL	Biaya perbaikan yang dikeluarkan
catatan	TEXT	Catatan hasil perbaikan

NOTIFIKASI

Menyimpan semua notifikasi sistem yang dikirim ke pengguna, seperti tagihan jatuh tempo, konfirmasi pembayaran, dan update status perbaikan.

Atribut	Tipe	Keterangan
id_notifikasi	INT (PK)	Primary key notifikasi
id_pengguna	INT (FK)	Penerima notifikasi
judul	VARCHAR	Judul notifikasi
pesan	TEXT	Isi pesan notifikasi
tipe	ENUM	Tagihan / Pembayaran / Perbaikan / Sistem
sudah_dibaca	BOOLEAN	Status baca notifikasi
dikirim_pada	TIMESTAMP	Waktu notifikasi dikirim

Relasi Antar Entitas

Berikut adalah tabel lengkap relasi antar entitas beserta kardinalitasnya:

Entitas A	Kardinalitas	Entitas B	Keterangan
PENGGUNA	1 — 0..1	PENYEWA	Satu pengguna bisa menjadi penyewa
PENGGUNA	1 — 0..1	PEMILIK_KOS	Satu pengguna bisa menjadi pemilik
PEMILIK_KOS	1 — N	KOS	Satu pemilik memiliki banyak kos
KOS	1 — N	KAMAR	Satu kos terdiri dari banyak kamar
PENYEWA	1 — N	KONTRAK_SEWA	Satu penyewa bisa punya banyak kontrak (historis)
KAMAR	1 — N	KONTRAK_SEWA	Satu kamar bisa disewa
KONTRAK_SEWA	1 — N	PEMBAYARAN	Satu kontrak menghasilkan banyak tagihan bulanan
KAMAR	1 — N	LAPORAN_KERUSAKAN	Satu kamar bisa memiliki banyak laporan kerusakan
PENYEWA	1 — N	LAPORAN_KERUSAKAN	Satu penyewa bisa membuat banyak laporan
LAPORAN_KERUSA	AN1 — N	PEKERJAAN_PERBAIKA	Satu laporan bisa ditangani dalam beberapa kali perbaikan
PENGGUNA	1 — N	PEKERJAAN_PERBAIKA	Satu teknisi bisa mengerjakan banyak pekerjaan
Entitas A	Kardinalitas	Entitas B	Keterangan
PENGGUNA	1 — N	NOTIFIKASI	Satu pengguna menerima banyak notifikasi

4. Kardinalitas Relasi Antar Entitas

Kardinalitas mendefinisikan jumlah instance dari satu entitas yang dapat berelasi dengan instance dari entitas lain. Berikut adalah kardinalitas setiap relasi dalam ERD sistem informasi pengelolaan kos:

Entitas A	Relasi	Entitas B	Kardinalitas	Keterangan
PENGGUNA	sebagai	PENYEWA	1 : 0..1	Satu pengguna dapat memiliki nol atau satu data penyewa
PENGGUNA	sebagai	PEMILIK_KOS	1 : 0..1	Satu pengguna dapat memiliki nol atau satu data pemilik
PENGGUNA	menerima	NOTIFIKASI	1 : N	Satu pengguna menerima banyak notifikasi
PENGGUNA	dikerjakan oleh	PEKERJAAN_PERBAIKAN	1 : N	Satu teknisi dapat mengerjakan banyak pekerjaan
PEMILIK_KOS	memiliki	KOS	1 : N	Satu pemilik dapat memiliki banyak properti kos
KOS	terdiri dari	KAMAR	1 : N	Satu kos memiliki banyak kamar
PENYEWA	membuat	KONTRAK_SEWA	1 : N	Satu penyewa dapat memiliki banyak kontrak secara historis
KAMAR	digunakan di	KONTRAK_SEWA	1 : N	Satu kamar dapat disewa bergantian oleh penyewa berbeda
KONTRAK_SEWA	menghasilkan	PEMBAYARAN	1 : N	Satu kontrak menghasilkan banyak tagihan bulanan
PENYEWA	melaporkan	LAPORAN KERUSAKAN	1 : N	Satu penyewa dapat membuat banyak laporan kerusakan
KAMAR	memiliki	LAPORAN KERUSAKAN	1 : N	Satu kamar dapat memiliki banyak laporan kerusakan
LAPORAN KERUSAKAN	ditangani	PEKERJAAN_PERBAIKAN	1 : N	Satu laporan dapat ditangani dalam beberapa tahap pekerjaan

Keterangan Notasi Kardinalitas:

- **1 : 1** — Satu instance entitas A berelasi dengan tepat satu instance entitas B.
- **1 : 0..1** — Satu instance entitas A berelasi dengan nol atau satu instance entitas B (opsional).

- **1 : N** — Satu instance entitas A berelasi dengan satu atau lebih instance entitas B.
- **M : N** — Banyak instance entitas A berelasi dengan banyak instance entitas B.

5. Normalisasi Database

Normalisasi adalah proses mengorganisasi struktur tabel database untuk mengurangi redundansi data dan meningkatkan integritas data. Proses normalisasi dilakukan secara bertahap dari bentuk normal pertama (1NF) hingga bentuk normal ketiga (3NF).

First Normal Form (1NF)

Syarat: Setiap kolom harus bersifat atomik (tidak dapat dibagi lagi), tidak ada grup yang berulang dalam satu baris, dan setiap baris harus unik.

Pemeriksaan dan Temuan:

Setelah memeriksa seluruh tabel pada ERD, ditemukan satu pelanggaran 1NF pada tabel KAMAR:

- **Tabel KAMAR** — Atribut *fasilitas* berpotensi menyimpan banyak nilai sekaligus dalam satu kolom, misalnya: "AC, WiFi, Lemari, Kamar Mandi Dalam". Ini melanggar syarat atomisitas 1NF karena satu kolom seharusnya hanya menyimpan satu nilai tunggal.

Solusi:

Atribut *fasilitas* dipisahkan ke dalam tabel baru:

- **FASILITAS_KAMAR** (id_fasilitas PK, id_kamar FK, nama_fasilitas)

Sehingga setiap fasilitas disimpan sebagai baris terpisah. Relasi yang terbentuk: KAMAR 1—N FASILITAS_KAMAR.

Tabel lain (PENGGUNA, PENYEWA, PEMILIK_KOS, KOS, KONTRAK_SEWA, PEMBAYARAN, LAPORAN_KERUSAKAN, PEKERJAAN_PERBAIKAN, NOTIFIKASI) telah memenuhi 1NF karena semua atributnya bersifat atomik.

Second Normal Form (2NF)

Syarat: Telah memenuhi 1NF dan setiap atribut non-kunci harus bergantung penuh (*fully functional dependent*) pada seluruh primary key, bukan hanya sebagian (*partial dependency*).

Catatan: 2NF hanya relevan untuk tabel dengan *composite primary key* (primary key gabungan lebih dari satu kolom). Partial dependency terjadi ketika atribut non-kunci hanya bergantung pada sebagian dari composite key.

Pemeriksaan:

Semua tabel dalam ERD sistem pengelolaan kos menggunakan *surrogate key* tunggal sebagai primary key (id_pengguna, id_penyewa, id_kamar, id_kontrak, dan sebagainya). Tabel FASILITAS_KAMAR yang

baru ditambahkan pada tahap 1NF juga menggunakan surrogate key tunggal (id_fasilitas). Karena tidak ada tabel dengan composite primary key, maka tidak ada kemungkinan partial dependency.

Kesimpulan: Seluruh tabel telah memenuhi 2NF tanpa perubahan struktural tambahan.

Third Normal Form (3NF)

Syarat: Telah memenuhi 2NF dan tidak terdapat *transitive dependency*, yaitu atribut non-kunci tidak boleh bergantung pada atribut non-kunci lain (harus bergantung langsung pada primary key).

Pemeriksaan per tabel:

Tabel	Atribut yang Diperiksa	Transitive Dependency?	Solusi / Keterangan
KOS	jumlah_kamar	Ya — merupakan derived attribute, dapat dihitung dari COUNT(KAMAR)	Hapus kolom jumlah_kamar; hitung via query COUNT saat dibutuhkan
KONTRAK_SEWA	harga_disepakati	Tidak — harga bisa berbeda dari harga kamar (negosiasi), bergantung langsung pada id_kontrak	Tidak ada perubahan
PEMBAYARAN	denda	Tidak — menyimpan nilai historis hasil kalkulasi, bergantung langsung pada id_pembayaran	Tidak ada perubahan (retained untuk audit)
LAPORAN_KERUSAKAN	id_kamar, id penyewa	Tidak — laporan bisa berasal dari penyewa yang kontraknya sudah berakhir, kedua FK diperlukan	Tidak ada perubahan
Tabel lain	Semua atribut	Tidak — semua bergantung langsung pada PK masing-masing	Tidak ada perubahan

Perubahan hasil 3NF: Hapus kolom *jumlah_kamar* dari tabel KOS. Nilai tersebut diperoleh melalui: *SELECT COUNT(*) FROM KAMAR WHERE id_kos = ?*

Ringkasan Perubahan Normalisasi

Tahap	Perubahan	Keterangan
1NF	Tambah tabel FASILITAS_KAMAR	Memecah atribut multi-nilai fasilitas dari tabel KAMAR
2NF	Tidak ada perubahan	Semua tabel menggunakan surrogate key tunggal, tidak ada partial dependency

3NF	Hapus kolom jumlah_kamar dari tabel KOS	Nilai dapat dihitung melalui query COUNT; menyimpannya berisiko inkonsistensi
-----	---	---

Struktur Tabel Akhir Setelah Normalisasi (3NF)

Setelah melalui proses normalisasi, database terdiri dari **11 tabel** dengan struktur sebagai berikut:

No.	Nama Tabel	Atribut
1.	PENGGUNA	id_pengguna (PK), nama, email, no_hp, password_hash, peran, dibuat_pada
2.	PEMILIK_KOS	id_pemilik (PK), id_pengguna (FK), nama_usaha, alamat_usaha, no_rekening
3.	PENYEWA	id_penyewa (PK), id_pengguna (FK), no_ktp, pekerjaan, kontak_darurat
4.	KOS	id_kos (PK), id_pemilik (FK), nama_kos, alamat, deskripsi [Kolom jumlah_kamar dihapus — diperoleh via SELECT COUNT(*) FROM KAMAR WHERE id_kos = ?]
5.	KAMAR	id_kamar (PK), id_kos (FK), nomor_kamar, tipe, harga_per_bulan, status [Kolom fasilitas dipindahkan ke tabel FASILITAS_KAMAR]
6.	FASILITAS_KAMAR *)	id_fasilitas (PK), id_kamar (FK), nama_fasilitas
7.	KONTRAK_SEWA	id_kontrak (PK), id_penyewa (FK), id_kamar (FK), tgl_mulai, tgl_selesai, harga_disepakati, status_kontrak
8.	PEMBAYARAN	id_pembayaran (PK), id_kontrak (FK), tgl_tagihan, tgl_bayar, jumlah, denda, metode_bayar, status_bayar
9.	LAPORAN_KERUSAKAN	id_laporan (PK), id_kamar (FK), id_penyewa (FK), judul, deskripsi, prioritas, status, dilaporkan_pada
10.	PEKERJAAN_PERBAIKAN	id_pekerjaan (PK), id_laporan (FK), id_teknisi (FK), tgl_mulai, tgl_selesai, biaya, catatan
11.	NOTIFIKASI	id_notifikasi (PK), id_pengguna (FK), judul, pesan, tipe, sudah_dibaca, dikirim_pada

*) Tabel baru hasil normalisasi 1NF

7. Implementasi Desain Database ke MySQL / MariaDB (phpMyAdmin)

8. Persiapan Database

Sebelum menjalankan perintah DDL, buat database terlebih dahulu dan aktifkan penggunaannya.

```
DROP DATABASE IF EXISTS db_pengelolaan_kos; CREATE DATABASE db_pengelolaan_kos
CHARACTER SET utf8mb4;

USE db_pengelolaan_kos;
```

9. Cara Menjalankan di phpMyAdmin

1. Buka phpMyAdmin di browser: <http://localhost/phpmyadmin>
2. Pilih tab **SQL** pada halaman utama
3. Salin seluruh isi file `implementasi_kos.sql` dan tempelkan ke kolom SQL
4. Klik tombol **Go** — semua tabel akan terbuat secara otomatis
5. Verifikasi: database `db_pengelolaan_kos` dan 11 tabel akan muncul di panel kiri

10. Struktur Tabel (DDL)

Database terdiri dari **11 tabel** hasil normalisasi 3NF. Setiap tabel menggunakan storage engine **InnoDB** agar foreign key dan transaksi ACID aktif.

Tabel 1 — PENGGUNA

Entitas pusat. Semua aktor (Pemilik, Admin, Penyewa, Teknisi) terdaftar di sini. Kolom peran menggunakan tipe ENUM untuk membatasi nilai yang valid.

```
CREATE TABLE PENGGUNA (
    id_pengguna      INT AUTO_INCREMENT
    PRIMARY KEY,    nama      VARCHAR(100) NOT
    NULL,           email      VARCHAR(150) NOT NULL
    UNIQUE,         no_hp      VARCHAR(20) NOT NULL,
    password_hash    VARCHAR(255) NOT NULL,
    peran            ENUM('Pemilik','Admin','Penyewa','Teknisi') NOT NULL,
    dibuat_pada      DATETIME DEFAULT CURRENT_TIMESTAMP )
ENGINE=InnoDB;
```

Tabel 2 — PEMILIK_KOS

Ekstensi dari PENGGUNA khusus untuk pemilik properti. Constraint UNIQUE (id_pengguna) memastikan relasi 1-to-1 dengan PENGGUNA.

```
CREATE TABLE PEMILIK_KOS (
    id_pemilik INT AUTO_INCREMENT PRIMARY KEY, id_pengguna INT NOT NULL
    UNIQUE, nama_usaha VARCHAR(150) NOT NULL, alamat_usaha TEXT NOT
    NULL, no_rekening VARCHAR(50) NOT NULL, FOREIGN KEY (id_pengguna)
    REFERENCES PENGGUNA(id_pengguna) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Tabel 3 — PENYEWA

Ekstensi dari PENGGUNA untuk penyewa. Menyimpan data tambahan seperti nomor KTP dan kontak darurat.

```
CREATE TABLE PENYEWA (
    id_penyewa INT AUTO_INCREMENT PRIMARY KEY, id_pengguna INT NOT
    NULL UNIQUE, no_ktp CHAR(16) NOT NULL UNIQUE, pekerjaan
    VARCHAR(100) NOT NULL, kontak_darurat VARCHAR(15) NOT NULL, FOREIGN
    KEY (id_pengguna) REFERENCES PENGGUNA(id_pengguna) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Tabel 4 — KOS

Properti kos milik pemilik. Kolom jumlah_kamar **dihapus** (hasil normalisasi 3NF) karena nilainya dapat dihitung via COUNT().

```
CREATE TABLE KOS (
    id_kos INT AUTO_INCREMENT PRIMARY KEY, id_pemilik INT NOT NULL,
    nama_kos VARCHAR(150) NOT NULL, alamat TEXT NOT NULL, deskripsi TEXT,
    FOREIGN KEY (id_pemilik) REFERENCES PEMILIK_KOS(id_pemilik) ON DELETE
    CASCADE
) ENGINE=InnoDB;
```

Tabel 5 — KAMAR

Unit kamar dalam satu kos. Kolom fasilitas **dipindahkan** ke tabel FASILITAS_KAMAR (hasil normalisasi 1NF). Constraint UNIQUE (id_kos, nomor_kamar) mencegah duplikasi nomor kamar dalam satu kos.

```
CREATE TABLE KAMAR (
    id_kamar INT AUTO_INCREMENT PRIMARY
    KEY, id_kos INT NOT NULL,
    nomor_kamar VARCHAR(10) NOT NULL, tipe
```

```

VARCHAR(50) NOT NULL,  harga_per_bulan
DECIMAL(12,2) NOT NULL,

    status      ENUM('Tersedia','Terisi','Perbaikan') NOT NULL DEFAULT 'Tersedia',  UNIQUE
(id_kos, nomor_kamar),

    FOREIGN KEY (id_kos) REFERENCES KOS(id_kos) ON DELETE CASCADE

) ENGINE=InnoDB;

```

Tabel 6 — FASILITAS_KAMAR (tabel baru — hasil normalisasi 1NF)

Memecah atribut multi-nilai fasilitas dari tabel KAMAR. Setiap fasilitas disimpan sebagai satu baris terpisah.

```

CREATE TABLE FASILITAS_KAMAR (

    id_fasilitas INT AUTO_INCREMENT PRIMARY KEY,  id_kamar  INT
NOT NULL,  nama_fasilitas VARCHAR(100) NOT NULL,  FOREIGN KEY
(id_kamar) REFERENCES KAMAR(id_kamar) ON DELETE CASCADE

) ENGINE=InnoDB;

```

Tabel 7 — KONTRAK_SEWA

Penghubung antara PENYEWA dan KAMAR. Menyimpan detail perjanjian sewa termasuk harga yang disepakati (bisa berbeda dari harga kamar karena negosiasi).

```

CREATE TABLE KONTRAK_SEWA (

    id_kontrak  INT AUTO_INCREMENT PRIMARY
KEY,  id_penyewa  INT NOT NULL,  id_kamar
INT NOT NULL,  tgl_mulai  DATE NOT NULL,
tgl_selesai  DATE NOT NULL,  harga_disepakati
DECIMAL(12,2) NOT NULL,

    status_kontrak  ENUM('Aktif','Selesai','Dibatalkan') NOT NULL DEFAULT 'Aktif',
FOREIGN KEY (id_penyewa) REFERENCES PENYEWA(id_penyewa),  FOREIGN KEY
(id_kamar) REFERENCES KAMAR(id_kamar)

) ENGINE=InnoDB;

```

Tabel 8 — PEMBAYARAN

Mencatat setiap tagihan dan realisasi pembayaran sewa per periode kontrak.

```

CREATE TABLE PEMBAYARAN (

    id_pembayaran INT AUTO_INCREMENT
PRIMARY KEY,  id_kontrak  INT NOT NULL,

```

```

tgl_tagihan DATE NOT NULL, tgl_bayar
DATE,

jumlah DECIMAL(12,2) NOT NULL, denda
DECIMAL(12,2) NOT NULL DEFAULT 0, metode_bayar
ENUM('Transfer','Tunai','QRIS','Lainnya'),
status_bayar ENUM('Belum Bayar','Lunas','Terlambat') NOT NULL DEFAULT 'Belum Bayar',
FOREIGN KEY (id_kontrak) REFERENCES KONTRAK_SEWA(id_kontrak)

) ENGINE=InnoDB;

```

Tabel 9 — LAPORAN_KERUSAKAN

Mencatat laporan kerusakan fasilitas yang diajukan oleh penyewa.

```

CREATE TABLE LAPORAN_KERUSAKAN (

id_laporan INT AUTO_INCREMENT PRIMARY KEY,

id_kamar INT NOT NULL,
id_penyewa INT NOT NULL, judul
VARCHAR(200) NOT NULL, deskripsi
TEXT NOT NULL,

prioritas ENUM('Rendah','Sedang','Tinggi') NOT NULL DEFAULT 'Sedang', status
ENUM('Dilaporkan','Dikerjakan','Selesai') NOT NULL DEFAULT 'Dilaporkan', dilaporkan_pada
DATETIME DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (id_kamar) REFERENCES
KAMAR(id_kamar), FOREIGN KEY (id_penyewa) REFERENCES PENYEWA(id_penyewa)

) ENGINE=InnoDB;

```

Tabel 10 — PEKERJAAN_PERBAIKAN

Mencatat penugasan teknisi dan hasil pekerjaan untuk setiap laporan kerusakan.

```

CREATE TABLE PEKERJAAN_PERBAIKAN (

id_pekerjaan INT AUTO_INCREMENT
PRIMARY KEY, id_laporan INT NOT NULL,
id_teknisi INT NOT NULL, tgl_mulai DATE
NOT NULL, tgl_selesai DATE,

biaya DECIMAL(12,2) NOT NULL DEFAULT 0, catatan TEXT,
FOREIGN KEY (id_laporan) REFERENCES
LAPORAN_KERUSAKAN(id_laporan), FOREIGN KEY (id_teknisi)
REFERENCES PENGGUNA(id_pengguna)

) ENGINE=InnoDB;

```

Tabel 11 — NOTIFIKASI

Menyimpan semua notifikasi sistem yang dikirim ke pengguna.

```
CREATE TABLE NOTIFIKASI (  
    id_notifikasi INT AUTO_INCREMENT  
    PRIMARY KEY, id_pengguna INT NOT NULL,  
    judul VARCHAR(200) NOT NULL, pesan  
    TEXT NOT NULL,  
  
    tipe ENUM('Tagihan','Pembayaran','Kerusakan','Kontrak','Umum') NOT NULL,  
    sudah_dibaca TINYINT(1) NOT NULL DEFAULT 0, dikirim_pada DATETIME DEFAULT  
    CURRENT_TIMESTAMP, FOREIGN KEY (id_pengguna) REFERENCES  
    PENGGUNA(id_pengguna) ON DELETE CASCADE  
  
    ) ENGINE=InnoDB;
```

11. Perintah SQL: INSERT, UPDATE, DELETE

12.INSERT — Pengisian Data Awal

Data dimasukkan sesuai urutan ketergantungan foreign key: dimulai dari PENGGUNA sebagai tabel induk, kemudian tabel-tabel turunannya.

INSERT PENGGUNA

```
INSERT INTO PENGGUNA (nama, email, no_hp, password_hash, peran) VALUES  
  
('Tanaka Hiroshi', 'Tanaka@email.com', '081234567890', SHA2('pass123',256), 'Pemilik'),  
  
('Satou Kenji', 'Satou@email.com', '082345678901', SHA2('pass123',256), 'Admin'),  
  
('Nakamura Daiki', 'Nakamura@email.com', '083456789012', SHA2('pass123',256), 'Penyewa'),  
  
('Fujimoto Ryouta', 'Fujimoto@email.com', '084567890123', SHA2('pass123',256), 'Penyewa'),  
  
('Aizawa shun', 'Aizawa@email.com', '085678901234', SHA2('pass123',256), 'Teknisi');
```

INSERT PEMILIK_KOS dan PENYEWA

```
INSERT INTO PEMILIK_KOS (id_pengguna, nama_usaha, alamat_usaha, no_rekening) VALUES  
  
(1, 'Kos Tanaka property', 'Jl. Merdeka No. 7, Surabaya', '1234567890');  
  
INSERT INTO PENYEWA (id_pengguna, no_ktp, pekerjaan, kontak_darurat) VALUES  
  
(3, '3578010101990001', 'Mahasiswa', '081111111111'),  
  
(4, '3578010202990002', 'Karyawan Swasta', '082222222222');
```

INSERT KOS, KAMAR, dan FASILITAS_KAMAR

```
INSERT INTO KOS (id_pemilik, nama_kos, alamat, deskripsi) VALUES (1, 'Kos Melati Indah', 'Jl. Raya Darmo No. 25, Surabaya', 'Kos lokasi strategis.');
```

```
INSERT INTO KAMAR (id_kos, nomor_kamar, tipe, harga_per_bulan, status) VALUES (1, 'A01', 'Standar', 750000, 'Terisi'), (1, 'A02', 'Standar', 750000, 'Tersedia'), (1, 'B01', 'Deluxe', 1200000, 'Terisi'), (1, 'B02', 'Deluxe', 1200000, 'Perbaikan');
```

```
INSERT INTO FASILITAS_KAMAR (id_kamar, nama_fasilitas) VALUES (1, 'Kasur'), (1, 'Lemari'), (1, 'WiFi'), (2, 'Kasur'), (2, 'Lemari'), (2, 'WiFi'), (3, 'Kasur'), (3, 'Lemari'), (3, 'AC'), (3, 'WiFi'), (3, 'Kamar Mandi Dalam'), (4, 'Kasur'), (4, 'Lemari'), (4, 'AC'), (4, 'WiFi'), (4, 'TV');
```

INSERT KONTRAK_SEWA dan PEMBAYARAN

```
INSERT INTO KONTRAK_SEWA (id_penyewa, id_kamar, tgl_mulai, tgl_selesai, harga_disepakati, status_kontrak) VALUES (1, 1, '2025-01-01', '2025-12-31', 750000, 'Aktif'), (2, 3, '2025-02-01', '2026-01-31', 1150000, 'Aktif');
```

```
INSERT INTO PEMBAYARAN (id_kontrak, tgl_tagihan, tgl_bayar, jumlah, denda, metode_bayar, status_bayar) VALUES (1, '2025-01-01', '2025-01-01', 750000, 0, 'Transfer', 'Lunas'), (1, '2025-02-01', '2025-02-03', 750000, 0, 'Transfer', 'Lunas'), (1, '2025-03-01', '2025-03-10', 750000, 37500, 'QRIS', 'Terlambat'), (1, '2025-04-01', NULL, 750000, 0, NULL, 'Belum Bayar'), (2, '2025-02-01', '2025-02-01', 1150000, 0, 'Transfer', 'Lunas'), (2, '2025-03-01', '2025-03-01', 1150000, 0, 'Transfer', 'Lunas'), (2, '2025-04-01', NULL, 1150000, 0, NULL, 'Belum Bayar');
```

INSERT LAPORAN_KERUSAKAN, PEKERJAAN_PERBAIKAN, dan NOTIFIKASI

```
INSERT INTO LAPORAN_KERUSAKAN (id_kamar, id_penyewa, judul, deskripsi, prioritas, status) VALUES
```

(1,1,'Kran air bocor','Kran wastafel menetes meski sudah diputar rapat.','Sedang','Selesai'), (3,2,'AC tidak dingin','AC tidak mengeluarkan udara dingin sejak 2 hari.','Tinggi','Dikerjakan');

INSERT INTO PEKERJAAN_PERBAIKAN

(id_laporan,id_teknisi,tgl_mulai,tgl_selesai,biaya,catatan) VALUES

(1,5,'2025-03-06','2025-03-06',50000,'Kran diganti baru.'),

(2,5,'2025-04-03',NULL,0,'Menunggu pengiriman freon.');

INSERT INTO NOTIFIKASI (id_pengguna,judul,pesan,tipe,sudah_dibaca) VALUES

(3,'Tagihan April 2025','Tagihan Rp750.000 jatuh tempo 1 April 2025.','Tagihan',0),

(4,'Tagihan April 2025','Tagihan Rp1.150.000 jatuh tempo 1 April 2025.','Tagihan',0),

(3,'Pembayaran Dikonfirmasi','Pembayaran Maret Rp787.500 telah diterima.','Pembayaran',1);

Ringkasan Data yang Dimasukkan

Tabel	Jumlah Baris	Keterangan
PENGGUNA	5	1 Pemilik, 1 Admin, 2 Penyewa, 1 Teknisi
PEMILIK_KOS	1	Tanaka Hiroshi — Kos Tanaka Property
PENYEWA	2	Fujimoto Ryouta dan Nakamura Daiki
KOS	1	Kos Melati Indah
KAMAR	4	A01, A02 (Standar), B01, B02 (Deluxe)
FASILITAS_KAMAR	14	Fasilitas per kamar; menggantikan kolom multi-nilai
KONTRAK_SEWA	2	Kontrak aktif untuk Ahmad dan Dewi
PEMBAYARAN	7	Tagihan Jan–Apr; 1 terlambat, 1 belum dibayar
LAPORAN_KERUSAKAN	2	Kran bocor (selesai), AC tidak dingin (dikerjakan)
PEKERJAAN_PERBAIKAN	2	2 penugasan teknisi Joko
NOTIFIKASI	3	2 tagihan April, 1 konfirmasi pembayaran

13. UPDATE — Pembaruan Data

UPDATE 1 — Konfirmasi pembayaran April Fujimoto (terlambat, denda Rp37.500)

```
UPDATE PEMBAYARAN SET tgl_bayar = '2025-04-08',
metode_bayar = 'Transfer', status_bayar = 'Lunas',
denda = 37500 WHERE id_pembayaran = 4;
```

UPDATE 2 — Kamar B02 kembali Tersedia setelah renovasi selesai

```
UPDATE KAMAR SET status = 'Tersedia' WHERE id_kamar = 4;
```

UPDATE 3 — Perbaikan AC selesai, catat biaya dan tanggal selesai

```
UPDATE PEKERJAAN_PERBAIKAN
SET tgl_selesai = '2025-04-05', biaya = 350000,
catatan = 'Freon diisi ulang, AC kembali normal.'
WHERE id_pekerjaan = 2;
```

UPDATE 4 — Status laporan AC diperbarui menjadi Selesai

```
UPDATE LAPORAN_KERUSAKAN SET status = 'Selesai' WHERE id_laporan = 2;
```

UPDATE 5 — Harga kamar A02 naik mulai periode baru

```
UPDATE KAMAR SET harga_per_bulan = 800000 WHERE id_kamar = 2;
```


Ringkasan Operasi UPDATE

No.	Tabel	Kolom yang Diubah	Kondisi WHERE
1	PEMBAYARAN	tgl_bayar, metode_bayar, status_bayar, tanggal_due	id_pembayaran = 4
2	KAMAR	status	id_kamar = 4
3	PEKERJAAN_PERBAIKAN	tgl_selesai, biaya, catatan	id_pekerjaan = 2
4	LAPORAN_KERUSAKAN	status	id_laporan = 2
5	KAMAR	harga_per_bulan	id_kamar = 2

14.DELETE — Penghapusan Data

DELETE 1 — Hapus fasilitas TV dari kamar B02 (TV dipindahkan ke kamar lain)

```
DELETE FROM FASILITAS_KAMAR
```

```
WHERE id_kamar = 4 AND nama_fasilitas = 'TV';
```

DELETE 2 — Hapus notifikasi yang sudah dibaca oleh Ahmad

```
DELETE FROM NOTIFIKASI
```

```
WHERE id_pengguna = 3 AND sudah_dibaca = 1;
```

Catatan: Tabel FASILITAS_KAMAR dan NOTIFIKASI memiliki constraint ON DELETE CASCADE dari tabel induknya. Artinya, jika sebuah kamar atau pengguna dihapus, seluruh data turunannya akan terhapus secara otomatis tanpa perintah DELETE tambahan.

15.SELECT — Verifikasi Hasil Operasi

SELECT 1 — Cek semua pengguna

```
SELECT id_pengguna, nama, peran FROM PENGGUNA;
```

SELECT 2 — Kamar beserta daftar fasilitas

Menggunakan GROUP_CONCAT untuk menampilkan fasilitas dalam satu baris, menggantikan kolom multi-nilai yang dihapus saat normalisasi 1NF.

```
SELECT k.nomor_kamar, k.tipe, k.status,  
       GROUP_CONCAT(f.nama_fasilitas SEPARATOR ', ') AS fasilitas
```

```
FROM KAMAR k LEFT JOIN FASILITAS_KAMAR f ON
k.id_kamar = f.id_kamar

GROUP BY k.id_kamar;
```

SELECT 3 — Riwayat pembayaran lengkap

```
SELECT p.nama AS penyewa, km.nomor_kamar,
       pb.tgl_tagihan, pb.tgl_bayar, pb.jumlah, pb.denda, pb.status_bayar FROM
PEMBAYARAN pb

JOIN KONTRAK_SEWA ks ON pb.id_kontrak = ks.id_kontrak

JOIN PENYEWA pnyw ON ks.id_penyewa = pnyw.id_penyewa

JOIN PENGGUNA p ON pnyw.id_pengguna =
p.id_pengguna JOIN KAMAR km ON ks.id_kamar =
km.id_kamar

ORDER BY pb.tgl_tagihan;
```

SELECT 4 — Laporan kerusakan dan status perbaikan

```
SELECT lk.judul, lk.prioritas, lk.status,
       pg_p.nama AS penyewa, pg_t.nama AS teknisi, pp.biaya
FROM LAPORAN_KERUSAKAN lk

JOIN PENYEWA pnyw ON lk.id_penyewa = pnyw.id_penyewa

JOIN PENGGUNA pg_p ON pnyw.id_pengguna = pg_p.id_pengguna

LEFT JOIN PEKERJAAN_PERBAIKAN pp ON lk.id_laporan = pp.id_laporan

LEFT JOIN PENGGUNA pg_t ON pp.id_teknisi = pg_t.id_pengguna;
```

16. Aplikasi CRUD Menggunakan Python dan PHP

17. Pendahuluan

Aplikasi ini mengimplementasikan operasi CRUD (*Create, Read, Update, Delete*) pada dua tabel master: **PENGGUNA** dan **KOS**. Arsitektur yang digunakan adalah **Flask (Python)** sebagai **REST API backend** dan **PHP** sebagai **frontend** yang berkomunikasi dengan API melalui cURL. Tampilan antarmuka menggunakan Bootstrap 5.

Lapisan	Teknologi	Fungsi
Frontend	PHP 8 + Bootstrap 5	Tampilan CRUD, form modal, tabel da

Komunikasi	HTTP / JSON (cURL)	PHP memanggil endpoint REST Flask
Backend API	Python 3 + Flask	Memproses request, validasi, query MySQL
Database	MySQL / MariaDB	Menyimpan data PENGGUNA dan KOS

18. Strukt

ur File

Proyek_{kos_a}

pp/ ■■■

api/

■ ■■■ app.py Flask REST API (backend Python)

■ ■■■ requirements.txt Daftar dependensi Python

■■■ frontend/

■■■ config.php Konfigurasi URL API

■■■ helpers.php Fungsi cURL & flash message

■■■ index.php Redirect ke pengguna.php

■■■ pengguna.php Halaman CRUD Pengguna

■■■ kos.php Halaman CRUD Kos

■■■ layout/

■■■ header.php Sidebar + navbar bersama

■■■ footer.php Script Bootstrap

19.Backend — Flask REST API (Python)

File `api/app.py` menyediakan 12 endpoint REST untuk operasi CRUD pada tabel PENGGUNA dan KOS, ditambah satu endpoint helper untuk dropdown pemilik.

Konfigurasi Koneksi Database

```
DB_CONFIG = {
    "host": "localhost",
```

```

    "user": "root", "password": "", # sesuaikan
password MySQL

    "database": "db_pengelolaan_kos",
    "charset": "utf8mb4",

}

```

Daftar Endpoint API

Method	Endpoint	Fungsi
GET	/api/pengguna	Ambil semua data pengguna
GET	/api/pengguna/{id}	Ambil satu pengguna berdasarkan ID
POST	/api/pengguna	Tambah pengguna baru
PUT	/api/pengguna/{id}	Perbarui data pengguna
DELETE	/api/pengguna/{id}	Hapus pengguna
GET	/api/kos	Ambil semua kos (+ jumlah kamar via <
GET	/api/kos/{id}	Ambil satu kos berdasarkan ID
POST	/api/kos	Tambah kos baru
PUT	/api/kos/{id}	Perbarui data kos
DELETE	/api/kos/{id}	Hapus kos
GET	/api/pemilik	Daftar pemilik kos (untuk dropdown form)

Format Response API

Semua endpoint mengembalikan JSON dengan format standar:

```

// Response sukses { "status": "ok", "message":
"Berhasil", "data": { ... } }

// Response error

{ "status": "error", "message": "Pesan error" }

```

Contoh Endpoint — GET /api/pengguna

```

@app.route("/api/pengguna",
methods=["GET"]) def get_pengguna():

```

```

conn = get_db(); cur = conn.cursor(dictionary=True)
try:
    cur.execute(
        "SELECT id_pengguna, nama, email, no_hp, peran,
        dibuat_pada " "FROM PENGGUNA ORDER BY peran, nama")
    rows = cur.fetchall()
    for r in rows:
        if r.get("dibuat_pada"):
            r["dibuat_pada"] = str(r["dibuat_pada"])
    return ok(rows)
except Exception as e:
    return err(str(e))
finally:
    cur.close(); conn.close()

```

Contoh Endpoint — POST /api/pengguna

```

@app.route("/api/pengguna",
methods=["POST"]) def create_pengguna():
    d = request.get_json(force=True) or {} # Validasi field
    wajib = ["nama", "email", "no_hp", "password",
    "peran"]:
    if not d.get(f):
        return err(f"Field '{f}' wajib diisi") # Hash
        password dengan SHA-256
    pw_hash =
    hashlib.sha256(d["password"].encode()).hexdigest()
    conn =
    get_db(); cur = conn.cursor()
    try:
        cur.execute(
            "INSERT INTO PENGGUNA (nama, email, no_hp,
            password_hash, peran) "
            "VALUES (%s, %s, %s, %s, %s)",
            (d["nama"], d["email"],
            d["no_hp"], pw_hash, d["peran"]))
        conn.commit()
        return ok({"id_pengguna": cur.lastrowid, "Pegguna berhasil ditambahkan", 201})
    except mysql.connector.IntegrityError:
        return err("Email sudah terdaftar")
    finally:
        cur.close(); conn.close()

```

20. Frontend — PHP

config.php — Konfigurasi URL API

```

define('API_BASE_URL', 'http://localhost:5000/api');
define('APP_NAME', 'Sistem Pengelolaan Kos');

```

helpers.php — Wrapper cURL

Fungsi `api_call()` mengirim request HTTP ke Flask API menggunakan cURL dan mengembalikan response sebagai array PHP.

```
function api_call(string $endpoint, string $method = 'GET', array $body = []): array {  
    $url = API_BASE_URL . $endpoint;  
    $ch = curl_init($url);  
    curl_setopt_array($ch,  
    [    CURLOPT_RETURNTRANSFER  
=> true,  
        CURLOPT_CUSTOMREQUEST => $method,  
        CURLOPT_HTTPHEADER => ['Content-Type: application/json'],    ]);  
    if (in_array($method, ['POST', 'PUT']) && !empty($body))  
    {    curl_setopt($ch, CURLOPT_POSTFIELDS,  
        json_encode($body));    }  
  
    $raw = curl_exec($ch);    $code =  
    curl_getinfo($ch,    CURLINFO_HTTP_CODE);  
    curl_close($ch);    $result = json_decode($raw,  
    true) ?? [];    $result['http_code'] = $code;  
    return $result; }
```

Alur CRUD di pengguna.php dan kos.php

Kedua halaman menggunakan pola **PRG (Post-Redirect-Get)** untuk mencegah form ter-submit ulang saat halaman di-refresh.

```
[User klik aksi] → [Form POST ke PHP] → [PHP panggil API via cURL]  
→ [Flask query MySQL] → [Response JSON] → [PHP set flash]  
→ [Redirect ke halaman] → [Halaman GET tampilkan data]
```

21. Cara Menjalankan Aplikasi

Langkah 1 — Siapkan Database

Jalankan `implementasi_kos.sql` di phpMyAdmin (lihat Bagian 7).

Langkah 2 — Jalankan Backend Flask

```
cd kos_app/api pip install -r  
requirements.txt python app.py #  
Flask berjalan di http://localhost:5000
```

Langkah 3 — Jalankan Frontend PHP

1. Salin folder `kos_app/frontend/` ke `C:\xampp\htdocs\frontend\`
2. Pastikan Apache XAMPP sudah **Start**
3. Buka browser: `http://localhost/frontend/`

***Penting:** Flask (port 5000) dan Apache XAMPP (port 80) harus berjalan **bersamaan**.*

22. Fitur Aplikasi

Fitur	<code>pengguna.php</code>	<code>kos.php</code>
Tampil tabel data	✓	✓
Tambah data (modal form)	✓	✓
Edit data (modal form)	✓	✓
Hapus data (konfirmasi modal)	✓	✓
Flash notification sukses/gagal	✓	✓
Validasi field wajib	✓	✓
Dropdown relasi	Peran enum	Pemilik dari API
Info tambahan	Badge warna per peran	Jumlah kamar via COUNT
Password hashing (SHA-256)	✓	—